© The Author(s), under exclusive license to APress Media, LLC, part of Springer Nature 2025
S. K. Sahu, *Generative AI-Driven Application Development with Java*
https://doi.org/10.1007/979-8-8688-1609-3_13

13. Jlama and Friends: Hosting Models the Java Way

Satej Kumar Sahu[1]
(1) Berhampur, Odisha, India

The goal of this chapter is to host a large language model (LLM) using a Java-based LLM inference engine.

Introduction

In the previous chapters, you learned about using a prehosted LLM API like OpenAI or running a local inferencing engine like Ollama. Java has not been known as the platform to host an LLM through a native platform inferencing engine similar to the likes present in the Python environment. This is no longer true with the options of Jlama and DJL available in the Java ecosystem. In this chapter, you will explore what Jlama is and what value it adds to the Java ecosystem in the world of machine learning. While you are at it, you will get your hands dirty by hosting an LLM in Java and then using it to build a sample application.

To follow along with the examples, ensure you have the following prerequisites:

- Java 20 or later
- At least 16GB of RAM (especially for Q4 quantized model variants)

What Is Jlama?

The prevailing approach for building applications with LLMs involves relying on external services provided by specialized companies. These services typically grant access to large, general-purpose models, which leads to significant energy consumption and costs that scale with model size. Furthermore, this pattern raises privacy and security con-

cerns, as it's difficult to ensure how these service providers will use customer prompts, which may contain sensitive data.

Consequently, many organizations are investigating the alternative of training or fine-tuning smaller, more focused models tailored to their specific business needs. These models can then be run on-premises or in private clouds, giving companies greater control over their data and infrastructure.

However, integrating these specialized models into existing software infrastructure, which is often based on Java in enterprise environments, presents a challenge. While a traditional client-server architecture could be used (for example, by serving the model through an external server like Ollama and accessing it via REST APIs), this approach can be cumbersome for Java developers. They could achieve greater efficiency by consuming the model directly within Java, eliminating the need for additional tools.

Ultimately, embedding LLM interaction directly within the same Java process that runs the application simplifies the transition from local development to deployment. It also reduces the operational burden on IT, removing the need to manage a separate external server and potentially streamlining platform engineering strategies. This is where Jlama becomes a valuable solution.

Jlama is a modern LLM inference engine for Java. It is a significant development for Java developers working with LLMs. It's essentially a library that allows LLMs to run directly within a Java application. This is a departure from the traditional approach of using external servers to host and serve these models. Jlama supports many LLM families and implements advanced features, integrating well with frameworks like Quarkus through Langchain4j. By leveraging Java's Vector API, Jlama aims to provide efficient performance. The core idea is to enable Java developers to incorporate LLM capabilities into their applications in a more streamlined way, simplifying development, improving security, and potentially enhancing monitoring and observability.

Jlama is a modern Java library designed to bring the power of LLMs directly into Java applications. It supports a wide range of popular LLM architectures, including the Gemma, Llama (and its versions 2 and 3), Mistral and Mixtral, Qwen2, and IBM Granite families, as well as the foundational models GPT-2 and BERT. Furthermore, Jlama handles various tokenization methods (BPE and WordPiece), which are crucial for processing text data for LLMs.

Beyond model support, Jlama incorporates several advanced features to enhance performance and functionality. These include paged attention and mixture of experts, tech-

niques that improve efficiency and allow for more complex model architectures. It also supports tool calling, enabling LLMs to interact with external functions, and offers the ability to generate embeddings, which are numerical representations of text used in many NLP tasks. Jlama also provides classifier support for categorization tasks. It works with the Hugging Face SafeTensors format, a standard for storing and loading model weights, and handles both the model and tokenizer formats used by Hugging Face. To optimize computational efficiency, it supports various data types (F32, F16, BF16) and model quantization (Q8, Q4), which reduces model size and speeds up inference. Fast General Matrix Multiplication (GEMM) operations are implemented for efficient matrix calculations, and Jlama even offers distributed inference capabilities, allowing for scaling LLM processing across multiple devices.

To leverage these features, Jlama requires Java 20 or later, as it takes advantage of the Vector API for accelerated inference. This combination of broad model support and advanced features makes Jlama a powerful tool for integrating LLMs into Java-based applications.

Jlama is a Java library designed to enable LLM inference directly within Java applications. It supports a variety of LLM model families, including Llama, Mistral, Qwen2, and Granite, and implements features like function calling, model quantization, mixture of experts, and distributed inference. It is well-integrated with Quarkus via a dedicated LangChain4j extension. For performance, Jlama leverages Java's Vector API (currently a preview feature, expected to be fully supported in Java 25).

Essentially, Jlama allows LLMs to run within the same Java Virtual Machine (JVM) as the application, offering several advantages:

- **Simplified lifecycle management:** In cases where the LLM and the application have a tightly coupled lifecycle (e.g., a new application feature requires a model update, or model fine-tuning necessitates prompt changes), embedding the model simplifies versioning and tracking.
- **Faster development and prototyping:** Eliminating the need to set up, configure, and interact with an external server streamlines the development process for LLM-based Java applications.
- **Easier model testing:** Running LLM inference within the JVM simplifies the testing of different models and their integration during development.
- **Enhanced security:** By performing inference within the application's JVM, the need for REST API communication with an external LLM server is reduced, mitigating the risk of private data leaks and enabling finer-grained user authorization.

- **Monolithic application compatibility:** Jlama allows applications with LLM capabilities to be included in monolithic architectures without requiring significant changes.
- **Improved monitoring and observability:** Running LLM inference in Java simplifies monitoring and gathering statistics on LLM response reliability and speed.
- **Simplified debugging:** Developers can more easily debug LLM interactions by stepping through both application and Jlama code within the same environment.
- **Simplified distribution:** Embedding the LLM within the application package simplifies distribution (though this may be suitable only in specific scenarios).
- **Edge deployment suitability:** Self-contained LLM-capable Java applications are often better suited for edge environments than client-server architectures.
- **Auxiliary LLM embedding:** Applications that use multiple LLMs (e.g., agentic AI patterns, where a smaller LLM validates the output of a larger one) can benefit from embedding the smaller, auxiliary LLMs while serving the main LLM externally.

Jlama: Quick Start

Let's try Jlama. To start simply, it provides a command-line utility to use it. The CLI can be run with jbang, so install jbang first.

```
#Install jbang (or https://www.jbang.dev/download/)
curl -Ls https://sh.jbang.dev | bash -s - app setup
```

Once jbang is installed, install jlama using the following command:

```
#Install Jlama CLI (will ask if you trust the source)
jbang app install --force jlama@tjake
```

Let's explore the jlama command-line tool.

```
jlama -help
```

Output:

```
[jbang] Building jar for jlama.java...
Note: Annotation processing is enabled because one or more processors were found on the class path. A future release of javac may disable annotation processing unless at least one processor is specified by name (-processor), or a search path is specified (--processor-path, --processor-module-path), or annotation  processing is enabled explicitly (-proc:only, -proc:full).
```

```
 Use -Xlint:-options to suppress this message.
 Use -proc:none to disable annotation processing.
WARNING: Using incubator modules: jdk.incubator.vector
Usage:
jlama [COMMAND]
Description:
Jlama is a modern LLM inference engine for Java!
Quantized models are maintained at https://hf.co/tjake
Choose from the available commands:
Inference:
 chat              Interact with the specified model
 restapi           Starts a openai compatible rest api for interacting with this model
 complete          Completes a prompt using the specified model
Distributed Inference:
 cluster-coordinator  Starts a distributed rest api for a model using cluster workers
 cluster-worker       Connects to a cluster coordinator to perform distributed inference
Other:
 download          Downloads a HuggingFace model - use owner/name format
 list              Lists local models
 quantize          Quantize the specified model
```

Use Jlama to Host an LLM

Now that you have jlama installed, you can download a model from Hugging Face and chat with it. Note you also have prequantized models available at **https://hf.co/tjake**.

This provides the complete list of options jlama can be used for. For the moment, the subcommands that are of interest to us are chat and restapi. The **chat** subcommand allows us to interact with the specified LLM model on the command line. Let's explore it.

```
jlama chat –help
```

Output:

```
WARNING: Using incubator modules: jdk.incubator.vector
Usage:
jlama chat [OPTIONS] <model name>
Description:
Interact with the specified model
Parameters:
*    <model name>          The huggingface model owner/name pair
Command Options:
```

```
    --model-cache=ARG      The local directory for downloaded models
                             (default: models)
    --work-directory=ARG   Working directory for attention cache
    --temperature=ARG      Temperature of response [0,1] (default: 0.6)
    --tokens=ARG           Number of tokens to generate (default: 128000)
    --top-p=ARG            Controls how many different words the model
                             considers per token [0,1] (default: .9)
    --system-prompt=ARG    Change the default system prompt for this model
Download Options:
    --auto-download        Download the model if missing (default: false)
    --branch=ARG           The model branch to download from (default: main)
    --auth-token=ARG       HuggingFace auth token (for restricted models)
Advanced Options:
    --working-dtype=ARG    Working memory data type (default: F32)
    --working-qtype=ARG    Working memory quantization data type (default: I8)
    --threads=ARG          Number of threads to use (default: number of
                             physical cores)
    --quantize-to=ARG      Runtime Model quantization type
```

Let's run a simplified version of this command.

```
jlama chat tjake/Llama-3.2-1B-Instruct-JQ4 --auto-download
```

Output:

```
WARNING: Using incubator modules: jdk.incubator.vector
README.md 100% [=================================] 36/36KB (0:00:01 / 0:00:00)
config.json 100% [==============================] 877/877B (0:00:01 / 0:00:00)
model.safetensors 100% [=======================] 772/772MB (0:00:05 / 0:00:00)
tokenizer.json 100% [==============================] 9/9MB (0:00:01 / 0:00:00)
tokenizer_config.json 100% [====================] 54/54KB (0:00:01 / 0:00:00)
Using Native SIMD Operations (OffHeap)
Model type = Q4, Working memory type = F32, Quantized memory type = I8
Chatting with tjake/Llama-3.2-1B-Instruct-JQ4...
You:
```

We can start chatting at the “You” prompt by asking for a joke.

```
You: Tell me a joke.
Jlama: A man walked into a library and asked the librarian, "Do you have any books on
Pavlov's dogs?" The librarian replied, "It rings a bell, but I'm not sure if it's here or
not."
11 tokens/s (prompt), 7 tokens/s (gen)
You:
```

You can get out of the CLI prompt by using Ctrl+C. Next, you will explore the restapi subcommand.

```
# Run the openai chat api and UI on a model
jlama restapi tjake/Llama-3.2-1B-Instruct-JQ4 --auto-download
```

Output:

```
WARNING: Using incubator modules: jdk.incubator.vector
Using Native SIMD Operations (OffHeap)
Model type = Q4, Working memory type = F32, Quantized memory type = I8
Chat UI: http://localhost:8080
OpenAI Chat API: http://localhost:8080/chat/completions
 .   ___          _          __ _ _
/\\ / __'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\__ | '_ | '_| | '_ \/ _` | \ \ \ \
\\/  __)| |_)| | | | | || (_| |  ) ) ) )
 '  |___| .__|_| |_|_| |_\__, | / / / /
=========|_|==============|___/=/_/_/_/
:: Spring Boot ::                (v3.3.2)
2025-04-22T15:49:34.172Z  INFO 48593 --- [           main]
com.github.tjake.jlama.cli.JlamaCli      : Starting JlamaCli using Java 21.0.7 with PID
48593 (/home/ubuntu/.m2/repository/com/github/tjake/jlama-cli/0.8.2/jlama-cli-
0.8.2.jar started by ubuntu in /home/ubuntu)
2025-04-22T15:49:34.175Z  INFO 48593 --- [           main]
com.github.tjake.jlama.cli.JlamaCli      : No active profile set, falling back to 1 default
profile: "default"
2025-04-22T15:49:35.836Z  WARN 48593 --- [           main]
io.undertow.websockets.jsr                : UT026010: Buffer pool was not set on
WebSocketDeploymentInfo, the default pool will be used
2025-04-22T15:49:35.861Z  INFO 48593 --- [           main]
io.undertow.servlet                       : Initializing Spring embedded WebApplicationContext
2025-04-22T15:49:35.863Z  INFO 48593 --- [           main]
w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext:
initialization completed in 1598 ms
2025-04-22T15:49:36.082Z  INFO 48593 --- [           main] io.undertow    : starting
server: Undertow - 2.3.13.Final
```

```
2025-04-22T15:49:36.093Z  INFO 48593 --- [          main] org.xnio    : XNIO version
3.8.8.Final
2025-04-22T15:49:36.105Z  INFO 48593 --- [          main] org.xnio.nio    : XNIO NIO
Implementation Version 3.8.8.Final
2025-04-22T15:49:36.317Z  INFO 48593 --- [          main]
org.jboss.threads                : JBoss Threads version 3.5.0.Final
2025-04-22T15:49:36.396Z  INFO 48593 --- [          main]
o.s.b.w.e.undertow.UndertowWebServer     : Undertow started on port 8080 (http)
with context path '/'
2025-04-22T15:49:36.486Z  INFO 48593 --- [          main]
o.s.b.a.w.s.WelcomePageHandlerMapping    : Adding welcome page: class path
resource [static/index.html]
2025-04-22T15:49:37.404Z  INFO 48593 --- [          main]
com.github.tjake.jlama.cli.JlamaCli      : Started JlamaCli in 3.849 seconds (process
running for 7.063)
```

From this output, you can observe three things:

1. It uses Spring Boot internally to create this UI and api endpoint.
2 The UI for the chat can be accessed at http://localhost:8080.
.
3 The OpenAI chat API proxy can be accessed at
· http://localhost:8080/chat/completions.

Let's visit the UI app at http://localhost:8080. Figure 13-1 shows the UI of the chat app.

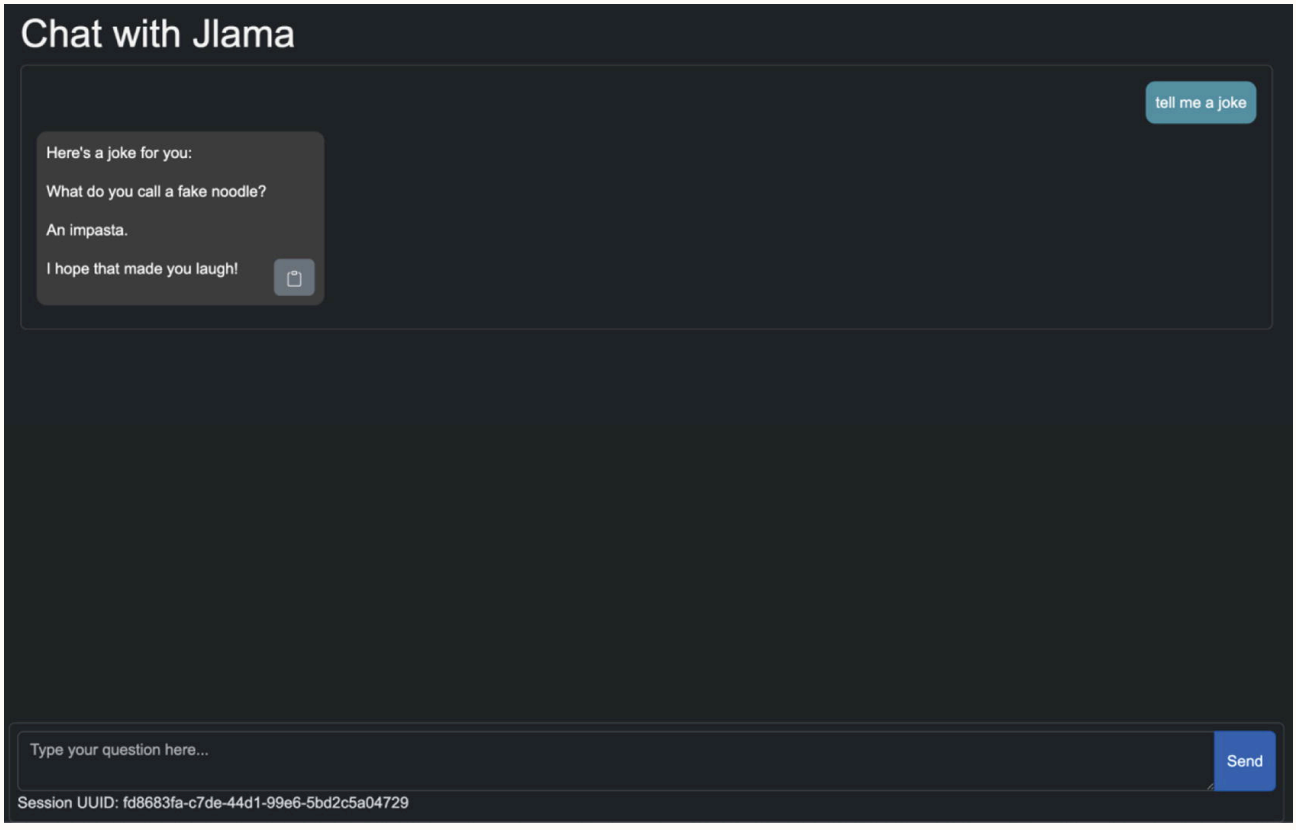


Figure 13-1 Jlama chat UI interface

This is handy to prototype and experiment quickly right in Java without any set up in Python. The next task for us is to see how to use it in a framework like Quarkus.

Quantize a Model Using Jlama

Quantization is a compression method that reduces high-precision values to lower-precision counterparts. In the context of LLMs, this involves lowering the precision of weights and activations to decrease memory usage. While this process can impact model performance, including accuracy, it is often a trade-off guided by specific use cases. In many scenarios, quantized models can deliver results comparable to full-precision versions, despite the reduced precision. Additionally, quantization enhances performance by lowering memory bandwidth demands and improving cache efficiency.

Jlama provides many options, and one of them is to quantize different LLMs. You can explore the different options to use it by typing the following command:

```
jlama quantize -help
```

Output:

```
Usage:
jlama quantize [OPTIONS] <model name> [<output>]
Description:
Quantize the specified model
Parameters:
*    <model name>        The huggingface model owner/name pair
    [<output>]          The output location
Command Options:
    --model-cache=ARG    The local directory for downloaded models (default:
                         models)
    --quantization=ARG   Model quantization type (default: Q4)
    --skip-layer=ARG     Layer name prefix to not quantize (default: norm)
    --drop-layer=ARG     Layer name prefix to drop
Download Options:
    --auto-download      Download the model if missing (default: false)
    --branch=ARG         The model branch to download from (default: main)
    --auth-token=ARG     HuggingFace auth token (for restricted models)
```

Currently, it provides support for quantization of following LLM family member as listed at https://github.com/tjake/Jlama/blob/main/jlama-core/src/main/java/com/github/tjake/jlama/model/ModelSupport.java. A sample of the code supporting different models at this point of time is shown here:

```
import com.github.tjake.jlama.model.bert.BertConfig;
import com.github.tjake.jlama.model.bert.BertModel;
import com.github.tjake.jlama.model.bert.BertTokenizer;
import com.github.tjake.jlama.model.gemma.GemmaConfig;
import com.github.tjake.jlama.model.gemma.GemmaModel;
import com.github.tjake.jlama.model.gemma.GemmaTokenizer;
import com.github.tjake.jlama.model.gemma2.Gemma2Config;
import com.github.tjake.jlama.model.gemma2.Gemma2Model;
import com.github.tjake.jlama.model.gpt2.GPT2Config;
import com.github.tjake.jlama.model.gpt2.GPT2Model;
import com.github.tjake.jlama.model.gpt2.GPT2Tokenizer;
import com.github.tjake.jlama.model.granite.GraniteConfig;
import com.github.tjake.jlama.model.granite.GraniteModel;
import com.github.tjake.jlama.model.llama.LlamaConfig;
import com.github.tjake.jlama.model.llama.LlamaModel;
import com.github.tjake.jlama.model.llama.LlamaTokenizer;
import com.github.tjake.jlama.model.mistral.MistralConfig;
import com.github.tjake.jlama.model.mistral.MistralModel;
import com.github.tjake.jlama.model.mixtral.MixtralConfig;
import com.github.tjake.jlama.model.mixtral.MixtralModel;
import com.github.tjake.jlama.model.qwen2.Qwen2Config;
import com.github.tjake.jlama.model.qwen2.Qwen2Model;
```

Now let's will experiment with Qwen2 model. You can start by visiting the Hugging Face page of Qwen at https://huggingface.co/Qwen, as shown in Figure 13-2.

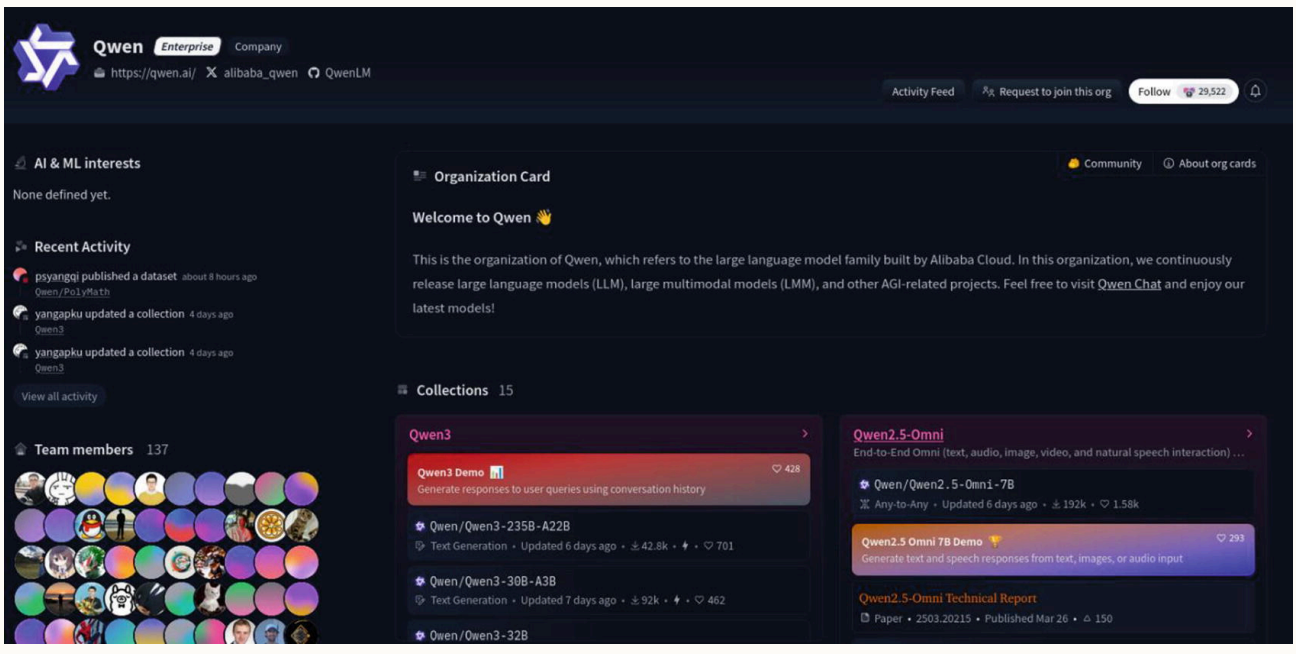


Figure 13-2 Qwen Hugging Face repository home page

On this page you will search for the Qwen2 family of models. Click the "Expand collections" button to see the Qwen2 model, as shown in Figure 13-3.

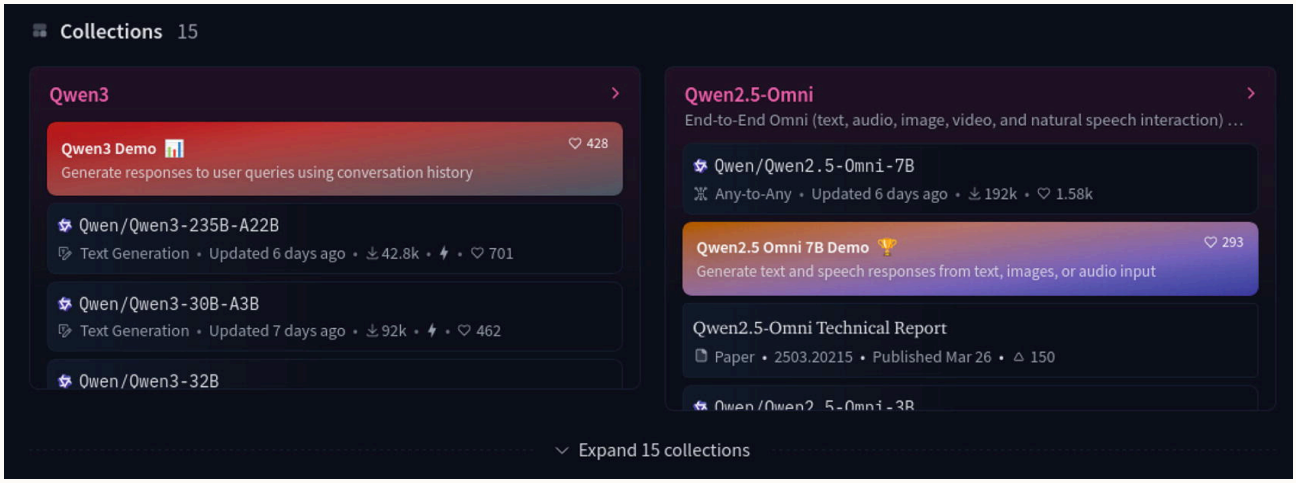


Figure 13-3 Qwen model collections

You can see the Qwen2 model list in the expanded collection in Figure 13-4.

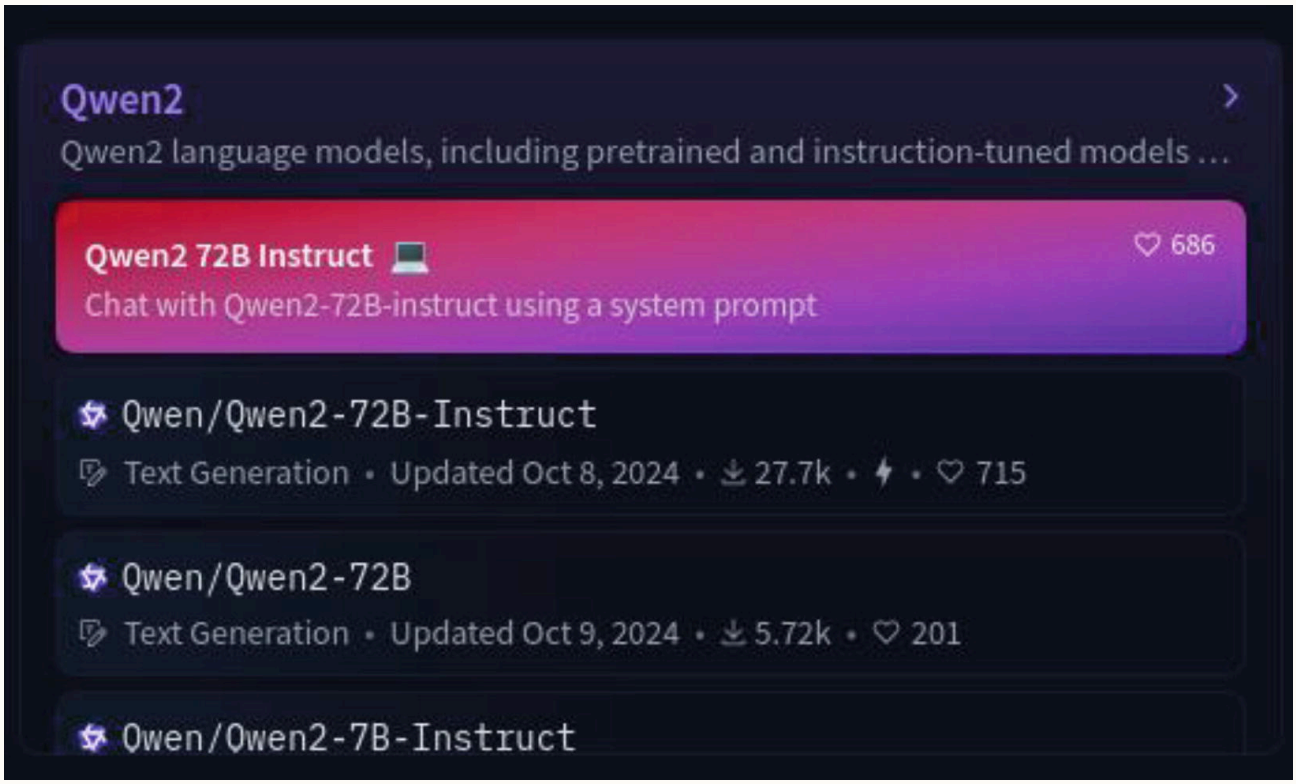


Figure 13-4 Qwen2 list of models

Click the Qwen2 Release link, which will take you to a separate page for Qwen2 based models, as shown in Figure 13-5.

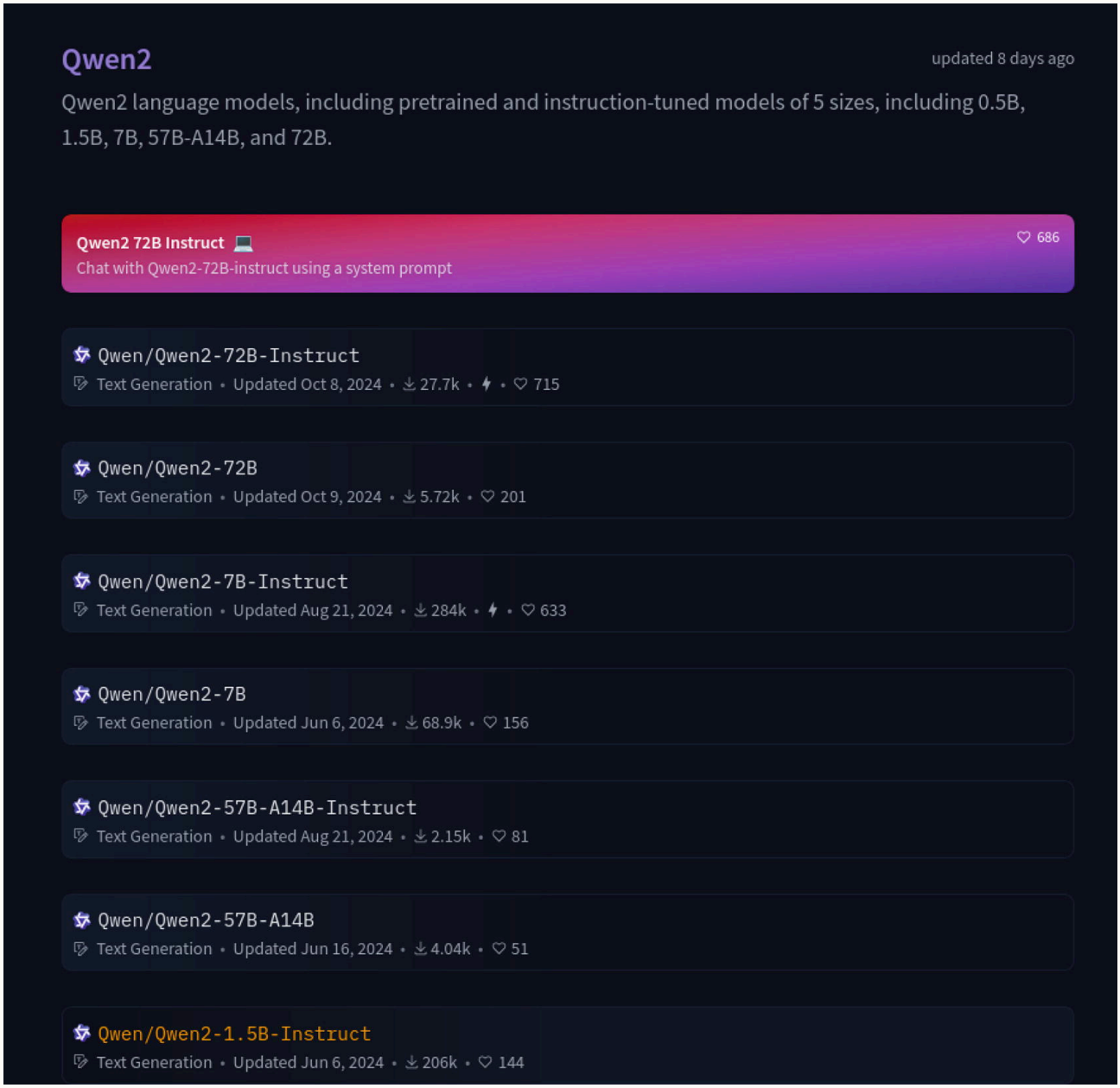


Figure 13-5 Qwen2 page for LLM models

Let's choose a simple small model for the use case, which is Qwen/Qwen2-0.5B. As shown in Figure 13-6, you will need to create an account if you don't already have one and in some cases like Gemma models from Google accept the conditions of use, as shown in Figure 13-6.

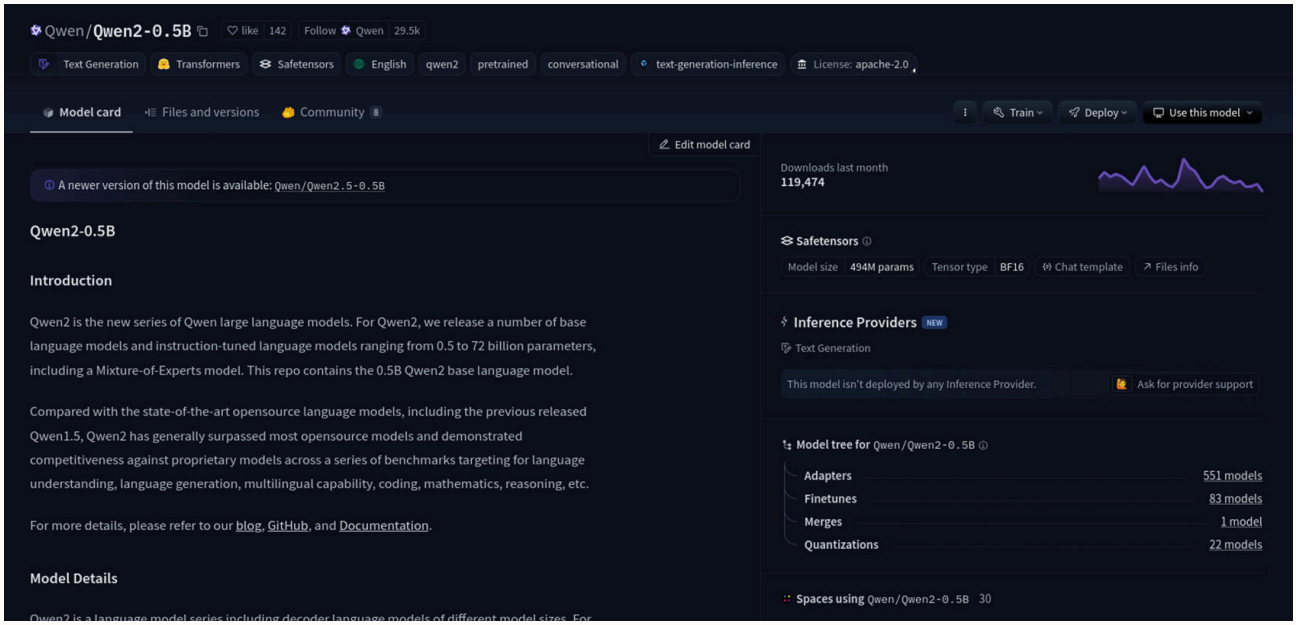


Figure 13-6 Qwen2-0.5B model page

To use this model, you will need an API token from the Hugging Face account, which has read permission. This can be done from the access tokens page in the profile drop-down list or by visiting https://huggingface.co/settings/tokens. Click the "Create new token" button and fill in the token form, as shown in Figure 13-7. Copy the token, which will be needed in the following section.

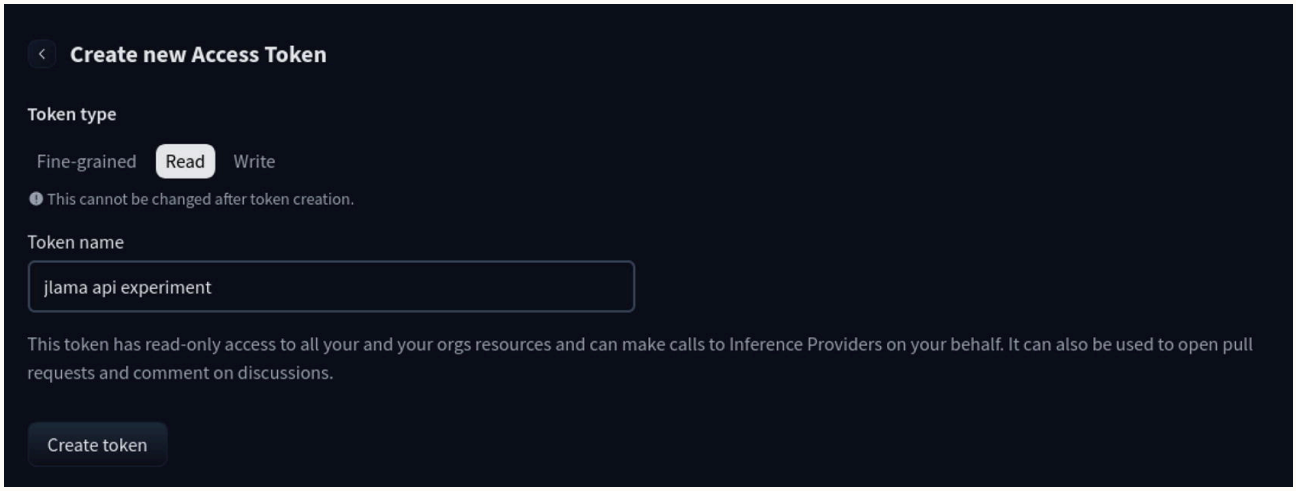


Figure 13-7 Hugging Face access token form

Now that you know which model you are going to quantize, let's proceed with using jlama command for this. The following will download the model relevant files and then quantize the model:

```
jlama quantize Qwen/Qwen2-0.5B --auto-download --auth-
token=YOUR_COPIED_ACCESS_TOKEN
```

Output:

NOTE: Picked up JDK_JAVA_OPTIONS: --add-modules jdk.incubator.vector --enable-preview WARNING: Using incubator modules: jdk.incubator.vector NOTE: Picked up JDK_JAVA_OPTIONS: --add-modules jdk.incubator.vector --enable-preview WARNING: Using incubator modules: jdk.incubator.vector README.md 100% [==========] 4/4KB (0:00:01 / 0:00:00) config.json 100% [==========] 661/661B (0:00:01 / 0:00:00) model.safetensors 100% [==========] 988/988MB (0:00:04 / 0:00:00) tokenizer.json 100% [==========] 7/7MB (0:00:01 / 0:00:00) tokenizer_config.json 0% [ ] 0/1KB (0:00:00 / ?) Skipping quantization of layer: model.norm.weight Skipping quantization of layer: model.layers.5.post_attention_layernorm.weight Skipping quantization of layer: model.layers.9.input_layernorm.weight tokenizer_config.json 100% [==========] 1/1KB (0:00:01 / 0:00:00) Skipping quantization of layer: model.layers.15.input_layernorm.weight Skipping quantization of layer: model.layers.22.input_layernorm.weight Skipping quantization of layer: model.layers.6.post_attention_layernorm.weight Skipping quantization of layer: model.layers.3.post_attention_layernorm.weight Skipping quantization of layer: model.layers.7.input_layernorm.weight Skipping quantization of layer: model.layers.10.post_attention_layernorm.weight Skipping quantization of layer: model.layers.17.input_layernorm.weight Skipping quantization of layer: model.layers.4.post_attention_layernorm.weight Skipping quantization of layer: model.layers.20.input_layernorm.weight Skipping quantization of layer: model.layers.18.input_layernorm.weight Skipping quantization of layer: model.layers.8.post_attention_layernorm.weight Skipping quantization of layer: model.layers.11.input_layernorm.weight Skipping quantization of layer: model.layers.9.post_attention_layernorm.weight Skipping quantization of layer: model.layers.11.post_attention_layernorm.weight Skipping quantization of layer: model.layers.4.input_layernorm.weight Skipping quantization of layer: model.layers.13.post_attention_layernorm.weight Skipping quantization of layer: model.layers.13.input_layernorm.weight Skipping quantization of layer: model.layers.2.input_layernorm.weight Skipping quantization of layer: model.layers.7.post_attention_layernorm.weight Skipping quantization of layer: model.layers.12.post_attention_layernorm.weight Skipping quantization of layer: model.layers.1.input_layernorm.weight Skipping quantization of layer: model.layers.15.post_attention_layernorm.weight Skipping quantization of layer: model.layers.14.input_layernorm.weight Skipping quantization of layer: model.layers.23.input_layernorm.weight Skipping quantization of layer: model.layers.8.input_layernorm.weight Skipping quantization of layer: model.layers.14.post_attention_layernorm.weight Skipping quantization of layer: model.layers.16.post_attention_layernorm.weight Skipping quantization of layer: model.layers.16.input_layernorm.weight Skipping quantization of layer: model.layers.21.input_layernorm.weight Skipping quantization of layer: model.layers.20.post_attention_layernorm.weight Skipping quantization of layer: model.layers.21.post_attention_layernorm.weight Skipping quantization of layer: model.layers.6.input_layernorm.weight Skipping quantization of layer: model.layers.2.post_attention_layernorm.weight Skipping quantization of layer:

```
model.layers.18.post_attention_layernorm.weight Skipping quantization of layer:
model.layers.5.input_layernorm.weight Skipping quantization of layer:
model.layers.19.input_layernorm.weight Skipping quantization of layer:
model.layers.10.input_layernorm.weight Skipping quantization of layer:
model.layers.22.post_attention_layernorm.weight Skipping quantization of layer:
model.layers.17.post_attention_layernorm.weight Skipping quantization of layer:
model.layers.0.post_attention_layernorm.weight Skipping quantization of layer:
model.layers.23.post_attention_layernorm.weight Skipping quantization of layer:
model.layers.3.input_layernorm.weight Skipping quantization of layer:
model.layers.12.input_layernorm.weight Skipping quantization of layer:
model.layers.1.post_attention_layernorm.weight Skipping quantization of layer:
model.layers.19.post_attention_layernorm.weight Quantized model written to:
/home/ubuntu/models/Qwen_Qwen2-0.5B-JQ4
```

This will create the quantized model in the directory where you ran the command inside another directory named models. Now, this model is much reduced in size and can be loaded with less memory usage. Let's use it with jlama to test it:

```
jlama chat Qwen/Qwen2-0.5B-JQ4
```

Output:

```
NOTE: Picked up JDK_JAVA_OPTIONS: --add-modules jdk.incubator.vector --enable-
preview WARNING: Using incubator modules: jdk.incubator.vector NOTE: Picked up
JDK_JAVA_OPTIONS: --add-modules jdk.incubator.vector --enable-preview WARNING:
Using incubator modules: jdk.incubator.vector Model type = Q4, Working memory
type = F32, Quantized memory type = F32
Chatting with Qwen/Qwen2-0.5B-JQ4...
You: Tell me about earth. Limit to a single sentence.
Jlama: Using Native SIMD Operations (OffHeap) Earth is a planet, a gas and rock ball in
space, a part of the solar system, and a part of the galaxy. It takes up about 99.98%
of the volume of the solar system and is about 40% of the total solar system's mass. It
is surrounded by a dense, stable, and molten layer of hydrogen, helium, and oxygen
called the atmosphere. The atmosphere is a thick layer that contains water vapor,
nitrogen, oxygen, and carbon dioxide, which are the building blocks of life. It is also
ruled by a protective ring of methane and nitrogen, known as the core, which contains
molten iron, nickel, and oxygen. The moon, Mercury, and Venus are also part of Earth's
system, but they are smaller and less visible.
```

This is some pretty interesting output from a quantized model, and you were able to achieve this without any deep knowledge of machine learning courtesy of the jlama engine.

Use Jlama with Quarkus to Build a Gen AI Application

By default, the sample project is configured to use a small Llama 2 model, quantized to 4 bits. When the application is initially built, Jlama automatically downloads this model from the Hugging Face repository to your local system. However, you can easily substitute this default model with any other compatible model to experiment with different LLMs. To do so, simply modify the quarkus.langchain4j.jlama.chat-model.model-name property within your application.properties file.

A dedicated Langchain4j extension provides strong integration between Jlama and Quarkus. Jlama's performance is enhanced by the Vector API, a preview feature in Java 23 that is likely to become a standard feature in Java 25.

You have already explored how the LangChain4j extension works with Quarkus in a previous chapters. Now you will extend it to use a Jlama-hosted LLM instead of the OpenAI API. I am using Java 21.0.7-tem using sdkman. Since Jlama uses preview features and the Vector API, make sure to set JDK_JAVA_OPTIONS as follows in your environment or in your pom.xml file:

```
export JDK_JAVA_OPTIONS="--add-modules jdk.incubator.vector --enable-preview"
```

There are three package dependencies you need to update in pom.xml, as shown here:

File: pom.xml

```
<dependency>
   <groupId>com.github.tjake</groupId>
   <artifactId>jlama-core</artifactId>
   <version>${jlama.version}</version>
</dependency>
<dependency>
   <groupId>io.quarkiverse.langchain4j</groupId>
   <artifactId>quarkus-langchain4j-jlama</artifactId>
   <version>0.26.2</version>
</dependency>
<dependency>
   <groupId>com.github.tjake</groupId>
   <artifactId>jlama-native</artifactId>
   <!-- supports linux-x86_64, macos-x86_64/aarch_64, windows-x86_64
      Use https://github.com/trustin/os-maven-plugin to detect os and arch -->
   <classifier>linux-x86_64</classifier>
   <version>${jlama.version}</version>
```

```
</dependency>
      ...
      ...
<configuration>
  <jvmArgs>--enable-preview --enable-native-access=ALL-UNNAMED</jvmArgs>
  <modules>
    <module>jdk.incubator.vector</module>
  </modules>
</configuration>
```

The next step is to update the application.properties file with some properties specific to Jlama such as the model name and the dimension value for the Redis store since the new model might have a different dimension for the embeddings than OpenAI, which you previously used.

File: src/main/resources/application.properties

```
quarkus.langchain4j.timeout=60s
quarkus.langchain4j.log-requests=true
quarkus.langchain4j.log-responses=true
quarkus.langchain4j.redis.dimension=384
quarkus.log.level=INFO
quarkus.package.jar.decompiler.enabled=true
quarkus.langchain4j.jlama.chat-model.model-name=tjake/Llama-3.2-3B-Instruct-JQ4
quarkus.langchain4j.jlama.chat-model.temperature=0
quarkus.langchain4j.jlama.log-requests=false
quarkus.langchain4j.jlama.log-responses=false
quarkus.langchain4j.chat-model.provider=jlama
quarkus.langchain4j.embedding-model.provider=jlama
```

These were all the changes needed. Seems simple, right? Let's boot up the Spring Boot application and test it.

Boot-up output:

```
[INFO] [io.quarkus.deployment.dev.DevModeCommandLineBuilder] Extension
io.quarkiverse.langchain4j:quarkus-langchain4j-jlama enables the C2 compiler which is
disabled by default in Dev mode for optimal performance.
[INFO] [io.quarkus.deployment.dev.DevModeCommandLineBuilder] Extension
io.quarkiverse.langchain4j:quarkus-langchain4j-jlama disables the Debug mode for
optimal performance. Debugging can still be enabled in the Quarkus plugin
configuration or with -Ddebug on the command line.
WARNING: Using incubator modules: jdk.incubator.vector
```

2025-04-22 20:25:46,366 INFO [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus Main Thread) Application starting...
2025-04-22 20:25:46,367 INFO [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus Main Thread) Inserting initial customer...
2025-04-22 20:25:46,534 INFO [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus Main Thread) **Initial customer inserted with ID: 1**
2025-04-22 20:25:46,542 INFO [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus Main Thread) **Documents read from catalog.csv**
2025-04-22 20:25:46,543 INFO [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus Main Thread) **Ingesting documents into Redis...**
2025-04-22 20:25:47,295 INFO [com.git.tja.jla.mod.AbstractModel] (Quarkus Main Thread) Model type = F32, Working memory type = F32, Quantized memory type = F32
2025-04-22 20:25:47,510 INFO [com.git.tja.jla.ten.ope.TensorOperationsProvider] (Quarkus Main Thread) Using Native SIMD Operations (OffHeap)
2025-04-22 20:25:49,409 INFO [io.dem.lan.sam.uti.ApplicationStartup] (Quarkus Main Thread) **Documents ingested into Redis.**
2025-04-22 20:25:49,500 INFO [io.quarkus] (Quarkus Main Thread) code-with-quarkus 1.0.0-SNAPSHOT on JVM (powered by Quarkus 3.21.3) started in 148.568s. Listening on: http://localhost:8080
2025-04-22 20:25:49,502 INFO [io.quarkus] (Quarkus Main Thread) Profile dev activated. Live Coding activated.
2025-04-22 20:25:49,503 INFO [io.quarkus] (Quarkus Main Thread) Installed features: [agroal, cdi, hibernate-orm, hibernate-orm-panache, hibernate-orm-rest-data-panache, jdbc-db2, langchain4j, langchain4j-jlama, langchain4j-openai, langchain4j-redis, narayana-jta, qute, redis-client, rest, rest-client, rest-client-jackson, rest-jackson, rest-links, smallrye-context-propagation, vertx]

Now let's test the curl request:

```
curl localhost:8080/review --header 'Content-Type: text/plain' --data 'This is a great product! I highly recommend it. Could you list all products from your shop catalog?'
```

System Output:

2025-04-22 20:26:59,656 INFO [io.dem.lan.sam.rag.RetrievalAugmentorExample] (executor-thread-1) Creating RetrievalAugmentorExample
2025-04-22 20:26:59,680 INFO [io.dem.lan.sam.rag.RetrievalAugmentorExample] (executor-thread-1) Retrieval AugmentorExample created
2025-04-22 20:27:01,718 INFO [com.git.tja.jla.mod.AbstractModel] (executor-thread-1) Model type = Q4, Working memory type = F32, Quantized memory type = I8

Curl Output:

{"evaluation":"POSITIVE","message":"Thank you for your review! We are glad you like our T-Shirt. We have listed all our products from our shop catalog below. **Please find the details of the product you have purchased below.** Product ID: 1001, Product Name: T-Shirt, Price: $15.99, Category: Clothing, Manufacturer: BrandA, SKU: TS-1001"}

As you can see from the output, there was no mention of the customer name, since this model does not support tooling like OpenAI did. The output is comparable to the previous output you received using the OpenAI API. There was an additional statement, "Please find the details of the product you have purchased below," which was unwarranted and can be further checked through prompt engineering. Also, instead of listing all the products, it listed only one. This is where you could leverage better prompts and also test with other models. The overall takeaway is that all of this is happening in the machine in Java.

Conclusion

In this chapter, you learned how the Java ecosystem has evolved to support native LLM inference through libraries like Jlama, eliminating the need for external Python-based servers or third-party APIs. By embedding Jlama directly into your Java application—whether via the simple CLI, through the OpenAI-compatible REST API, or seamlessly through Quarkus and Langchain4j—you gain full control over model lifecycle, data privacy, and operational complexity. You learned how to install and invoke Jlama using JBang, interact with models from the command line, and expose them as services, all while leveraging Java's Vector API for high-performance inference.

To put these concepts into practice, consider building small but meaningful projects such as:

- A local LLM-powered chatbot for customer support or internal tools
- A document summarization microservice using your own data
- An offline code generation assistant embedded in an IDE plugin
- A Quarkus-based REST API that performs Q&A over enterprise knowledge bases
- A CLI tool for text classification or sentiment analysis using fine-tuned models

These practical exercises will help solidify your understanding and expose real-world integration challenges—preparing you for more complex production scenarios.

Moreover, by integrating Jlama into a Quarkus-based generative AI application, you've experienced firsthand how minimal configuration changes in your pom.xml and application.properties files can switch your project from cloud-hosted LLMs to an on-premise solution—without sacrificing observability, security, or developer productivity. As you continue experimenting with different models, quantization strategies, and prompt-engineering techniques, you'll find that hosting LLMs in Java not only streamlines your development and deployment pipelines but also unlocks new possibilities for edge and monolithic architectures.

Armed with these tools and concepts, you're now ready to explore more advanced scenarios: fine-tuning smaller models for your domain, orchestrating hybrid open-source and commercial LLMs, and building complex agentic workflows—all within the familiar confines of the JVM. The next chapter will delve into exploring multimodal LLMs. Onward to mastering Java-native generative AI!

Previous chapter
< 12. Quarkus + LangChain4j: Lightning-Fast Gen AI

Next chapter
14. Seeing Is Believing: Multimodal LLMs and Image Hacking >